

Chapter 13 — Access

Keys, endpoints, and what a call costs

The cost thread has run through six chapters as a physical quantity. Here it becomes an invoice.

What a key authorises

- A key is a bearer credential. Whoever holds it is you, to the billing system.
- A paid chat subscription does not include API access. They are separate products with separate billing, and this surprises people every time.
- API usage is billed against prepaid credit, per token, with no monthly ceiling protecting you.

Key hygiene, in four rules

- **Environment variables, never source.** A key in a script is a key in your shell history and your editor's autosave.
- **Never in a shared repository.** Not in a private one either — private repositories get shared.
- **One key per person,** so a leak can be traced and revoked without stopping everyone.
- **Rotate on any doubt.** Rotation is cheap. Establishing that a key was not misused is not.

The group's local gateway issues per-user keys with per-user budgets. That is the mechanism that makes an exercise safe to run in a room.

Worked example: architecture from a price list

A published price break at a stated context length is enough to solve for something the vendor never published.

price step at a stated context length $\Rightarrow$ extra cost attributable to cached context

$\div$ tokens of context $\Rightarrow$ cost per cached token

$\div$ price per byte-hour $\Rightarrow$ bytes per token in the KV cache

Checked when this was recorded, the answer came out at roughly **1.7 kB per token** — then sanity-checked against plausible head dimensions and KV head counts, which is the step that turns arithmetic into evidence.

Why that example is in this course

- Order-of-magnitude estimation from public numbers, which is a skill this room already has and does not apply here.
- Dimensional analysis — the units carry the derivation, exactly as in Chapter 7.
- An independent check. The answer is only worth anything because it was tested against something it was not derived from.

TODO(verify) before delivery. The figure above is what the method produced when it was recorded. Pricing structures change, and a stale number presented as current is the failure this course spends three sessions warning about. **Re-run the arithmetic on the current price list.** The method is the transferable part; the number is not.

Local against frontier, side by side

- Same prompt. The group's own local model, and a frontier model. Both projected.
- This is the best available demonstration of everything in Chapter 5, on hardware the group already owns.
- Expect the local model to be worse in a specific, visible, instructive way — not uniformly worse.

TODO(capture) — needs the local machine reachable from the training room. If it is not, this becomes an instructor demo and keys are distributed afterwards. DECISIONS.md item 4.

Closing the cost thread

- **Metered pricing.** Per token, in and out, at different rates. Chapter 1's unit of meaning, priced.
- **Caching.** Re-sending the same standing context every turn is the cost Chapter 10 made you calculate. Caching is the mitigation, and it has its own price.
- **Batch discounts.** Latency traded for money, which is Chapter 7's roofline seen from the finance side.
- **Subscription against per-token.** Different economics, and the crossover depends on how you actually work, not on the headline rate.

Thread — cost, closed. Token as unit of meaning in Chapter 1, unit of price in Chapter 7, resident context in Chapter 10, orchestration premium in Chapter 11, and an invoice here.

Where this leaves us

- A credential model you can explain to whoever administers your grant.
- A worked example where public prices yielded an unpublished architectural number.
- A cost thread that started as a token and finished as a line item.

Session 2 ends here. Session 3 is your actual work.